

LINQ, DELEGATES SI EXPRESII LAMBDA

Tehnici avansate de programare

Nicoleta Scrimint



Facultatea de
MATEMATICĂ



Agenda



Made with Whimsical



Filtering

C#

 Copy

```
using (var context = new BloggingContext())
{
    var blogs = await context.Blogs
        .Where(b => b.Url.Contains("dotnet"))
        .ToListAsync();
}
```

Delegates

Intro

A **delegate** is a type that represents references to methods with a particular parameter list and return type.

- **Delegates** are used to pass methods as arguments to other methods.
- Event handlers are nothing more than methods that are invoked through delegates. You create a custom method, and a class, such as a service handler, can call your method when a certain event occurs.

```
public delegate int PerformCalculation(int x, int y);
```

Delegates

Pros



Delegates flexibility means you can programmatically change method calls or plug new code into existing classes.



This ability to refer to a method as a parameter makes delegates ideal for defining callback methods.



You can write a method that compares two objects in your application. That method can be used in a delegate for a sort algorithm. Because the comparison code is separate from the library, the sort method can be more general.

Delegates

Properties

Delegates have the following **properties**:

- ✓ Delegates are like C++ function pointers, but delegates are fully object-oriented, and unlike C++ pointers to member functions, delegates encapsulate both an object instance and a method.
- ✓ Delegates allow methods to be passed as parameters.
- ✓ Delegates can be used to define callback methods.
- ✓ Delegates can be chained together; for example, multiple methods can be called on a single event.
- ✓ Lambda expressions are a more concise way of writing inline code blocks.

Delegates

Example

```
public class LibrarySystem
{
    1 reference | 0 changes | 0 authors, 0 changes
    public void Welcome(string firstName, string lastName)
    {
        Console.WriteLine($"Welcome, {firstName} {lastName}");
    }
}
```

```
LibrarySystem library = new LibrarySystem();
```

```
Action<string, string> welcomeDelegate = library.Welcome;
welcomeDelegate("John", "Doe");
```



Delegates

Example

0 references | 0 changes | 0 authors, 0 changes

```
public void Login(string username, Action<string,string> welcome)
{
    Console.WriteLine($"Login with {username}");
    var firstName = "John";
    var lastName = "Doe";

    welcome(firstName, lastName);
}
```

```
LibrarySystem library = new LibrarySystem();

Action<string,string> welcomeDelegate = library.Welcome;

library.Login("john_doe", welcomeDelegate);
```



Delegates

Method

- A **delegate** can be associated with a **named method**. When you instantiate a delegate by using a named method, the method is passed as a parameter, for example:

```
// Declare a delegate.  
delegate void WorkCallback(int x);  
  
// Define a named method.  
void DoWork(int k) { /* ... */ }  
  
// Instantiate the delegate using the method as a parameter.  
WorkCallback d = obj.DoWork;
```



Delegates

Exercise

```
namespace Tema2Console
{
    delegate void CustomCallback(string s);

    class TestClass
    {
        static void Hello(string s)
        {
            Console.WriteLine($" Hello, {s}!");
        }

        static void Goodbye(string s)
        {
            Console.WriteLine($" Goodbye, {s}!");
        }
    }
}
```



Delegates

Exercise

```
static void Process()
{
    CustomCallback hiDel, byeDel, multiDel, multiMinusHiDel;

    hiDel = Hello;

    byeDel = Goodbye;

    multiDel = hiDel + byeDel;

    multiMinusHiDel = multiDel - hiDel;

    Console.WriteLine("Invoking delegate hiDel:");
    hiDel("A");

    Console.WriteLine("Invoking delegate byeDel:");
    byeDel("B");

    Console.WriteLine("Invoking delegate multiDel:");
    multiDel("C");

    Console.WriteLine("Invoking delegate multiMinusHiDel:");
    multiMinusHiDel("D");
}
```



Delegates

Exercise

```
Console.WriteLine("Invoking delegate hiDel:");
hiDel("A");

Console.WriteLine("Invoking delegate byeDel:");
byeDel("B");

Console.WriteLine("Invoking delegate multiDel:");
multiDel("C");

Console.WriteLine("Invoking delegate multiMinusHiDel:");
multiMinusHiDel("D");

/* Output:
Invoking delegate hiDel:
Hello, A!
Invoking delegate byeDel:
Goodbye, B!
Invoking delegate multiDel:
Hello, C!
Goodbye, C!
Invoking delegate multiMinusHiDel:
Goodbye, D!
*/
```



Delegates

Method

- Delegates constructed with a **named method** can encapsulate either:
 - a **static method** or an **instance method**.

```
// Declare a delegate
delegate void Callback();

class SampleClass
{
    public void InstanceMethod()
    {
        Console.WriteLine("A message from the instance method");
    }

    static public void StaticMethod()
    {
        Console.WriteLine("A message from the static method");
    }
}
```

```
class TestSampleClass
{
    static void Main()
    {
        var sc = new SampleClass();

        // Map the delegate to the instance method:
        Callback d = sc.InstanceMethod;
        d();

        // Map to the static method:
        d = SampleClass.StaticMethod;
        d();
    }
}

/* Output:
    A message from the instance method.
    A message from the static method.
*/
```



Delegates

Method

- **Named methods** are the only way to instantiate a delegate in earlier versions of C#.
 - However, in a situation where creating a new method is unwanted, C# enables you to instantiate a delegate and immediately specify a code block that the delegate will process when it is called.
 - The block can contain either a **lambda expression** or an **anonymous method**.

```
Func<int, int, int> sum = (a, b) => a + b;  
Console.WriteLine(sum(3, 4)); // output: 7
```

```
Func<int, int, int> sum = delegate (int a, int b) { return a + b; };  
Console.WriteLine(sum(3, 4)); // output: 7
```



Lambda expressions

Intro

- Lambda expressions, or just "lambdas" for short, were introduced in C# 3.0 as one of the core building blocks of Language Integrated Query (LINQ). They are just a more convenient syntax for using delegates.
- Currently we use C# 14 with .Net 10
 - Announcing .Net 10
 - The history of C#
- Delegates and lambdas



Lambda expressions

Intro

- You use a **lambda expression** to create an anonymous function.
 - Use the lambda declaration operator $\Rightarrow$ to separate the lambda's parameter list from its body.

A lambda expression can be of any of the following two forms:

- Expression lambda that has an **expression** as its body:

```
(input-parameters)  $\Rightarrow$  expression
```

- Statement lambda that has a **statement block** as its body:

```
(input-parameters)  $\Rightarrow$  { <sequence-of-statements> }
```



Lambda expressions

Delegate

- Any **lambda expression** can be converted to a **delegate type**.
 - The delegate type to which a lambda expression can be converted is defined by the types of its parameters and return value.

→ A lambda expression that has **two parameters** and **returns no value** can be converted to an **Action<T1,T2> delegate**.

↔ A lambda expression that has **one parameter** and **returns a value** can be converted to a **Func<T,TResult> delegate**.



Lambda expressions

Types

- Func<T, TResult>

```
Func<int, int> square = x => x * x;  
Console.WriteLine(square(5));  
// Output:  
// 25
```

- Expression

```
System.Linq.Expressions.Expression<Func<int, int>> e = x => x * x;  
Console.WriteLine(e);  
// Output:  
// x => (x * x)
```

- LINQ usage

```
int[] numbers = { 2, 3, 4, 5 };  
var squaredNumbers = numbers.Select(x => x * x);  
Console.WriteLine(string.Join(" ", squaredNumbers));  
// Output:  
// 4 9 16 25
```



Lambda expressions

Statement

- A **statement lambda** resembles an expression lambda except that its statements are enclosed in braces:

```
(input-parameters) => { <sequence-of-statements> }
```

Example:

```
Action<string> greet = name =>
{
    string greeting = $"Hello {name}!";
    Console.WriteLine(greeting);
};
greet("World");
// Output:
// Hello World!
```



Lambda expressions

Usage

- No parameters

```
Action line = () => Console.WriteLine();
```

- One input parameter

```
Func<double, double> cube = x => x * x * x;
```

- More input parameters

```
Func<int, int, bool> testForEquality = (x, y) => x == y;
```



Lambda expressions

Usage

- Specifying parameter types

```
Func<int, string, bool> isTooLong = (int x, string s) => s.Length > x;
```

- Parameters not used

```
Func<int, int, int> constant = (_, _) => 42;
```

- Parameter with default value (C# 12)

```
var IncrementBy = (int source, int increment = 1) => source + increment;  
  
Console.WriteLine(IncrementBy(5)); // 6  
Console.WriteLine(IncrementBy(5, 2)); // 7
```



Lambda expressions

Usage

- With params array as parameters

```
var sum = (params int[] values) =>
{
    int sum = 0;
    foreach (var value in values)
        sum += value;

    return sum;
};

var empty = sum();
Console.WriteLine(empty); // 0

var sequence = new[] { 1, 2, 3, 4, 5 };
var total = sum(sequence);
Console.WriteLine(total); // 15
```

Lambda expressions

Async

- You can easily create lambda expressions and statements that incorporate **asynchronous processing** by using the **async** and **await** keywords.

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        button1.Click += async (sender, e) =>
        {
            await ExampleMethodAsync();
            textBox1.Text += "\r\nControl returned to Click event handler.\n";
        };
    }

    private async Task ExampleMethodAsync()
    {
        // The following line simulates a task-returning asynchronous process.
        await Task.Delay(1000);
    }
}
```



Lambda expressions

Tuple
s

- The C# language provides built-in support for **tuples**. You can provide a tuple as an **argument** to a lambda expression, and your lambda expression can also **return** a tuple.

```
Func<(int, int, int), (int, int, int)> doubleThem = ns => (2 * ns.Item1, 2 * ns.Item2, 2 * ns.Item3);  
var numbers = (2, 3, 4);  
var doubledNumbers = doubleThem(numbers);  
Console.WriteLine($"The set {numbers} doubled: {doubledNumbers}");  
// Output:  
// The set (2, 3, 4) doubled: (4, 6, 8)
```

- Ordinarily, the fields of a tuple are named **Item1**, **Item2**, and so on. You can, however, define a tuple with named components, as the following example does.

```
Func<(int n1, int n2, int n3), (int, int, int)> doubleThem = ns => (2 * ns.n1, 2 * ns.n2, 2 * ns.n3);  
var numbers = (2, 3, 4);  
var doubledNumbers = doubleThem(numbers);  
Console.WriteLine($"The set {numbers} doubled: {doubledNumbers}");
```


Lambda expressions

Return type

- Typically, the **return type** of a lambda expression is **obvious** and **inferred**. For some expressions that doesn't work:

```
var choose = (bool b) => b ? 1 : "two"; // ERROR: Can't infer return type
```

- You can specify the **return type** of a lambda expression **before** the input parameters. When you specify an explicit return type, you must parenthesize the input parameters(C# 10):

```
var choose = object (bool b) => b ? 1 : "two"; // Func<bool, object>
```



Delegates – knowledge check

- Search on internet for “delegate and events training”
- Learn unit 5: Examine variance in delegates
- Unit 7: Knowledge check



- **Language-Integrated Query** (LINQ) is the name for a set of technologies based on the integration of query capabilities directly into the C# language.
- ✓ When you write **queries**, the most visible "language-integrated" part of LINQ is the query expression.
- ✓ Query expressions are written in a declarative query syntax.
- ✓ By using query syntax, you perform **filtering**, **ordering**, and **grouping** operations on data sources with a minimum of code.
- ✓ You use the same query expression patterns to query and **transform data** from any type of data source.



LINQ

Example

```
// Specify the data source.  
int[] scores = {97, 92, 81, 60};  
  
// Define the query expression.  
IEnumerable<int> scoreQuery =  
    from score in scores  
    where score > 80  
    select score;  
  
// Execute the query.  
foreach (var i in scoreQuery)  
{  
    Console.WriteLine(i + " ");  
}
```

```
....  
IEnumerable<int> scoreQuery = scores.Where(s => s > 80);
```





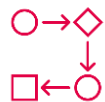
Query expressions query and transform data from any LINQ-enabled data source.
- For example, a single query can retrieve data from an SQL database and produce an XML stream as output.



Query expressions use many familiar C# language constructs, which make them easy to read.



The variables in a query expression are all strongly typed.



A query isn't executed until you iterate over the query variable, for example in a foreach statement.



In-memory data

- There are **two ways** you **enable LINQ** querying of in-memory data.
 - If the data is of a type that **implements `IEnumerable<T>`**, you query the data by using LINQ to Objects.
 - **You define LINQ standard query operator methods**, either in that type or as extension methods for that type.



Remote data

- The best option for enabling LINQ querying of a remote data source is to implement the **`IQueryable<T>`** interface.

LINQ

Actions

All LINQ query operations consist of three distinct **actions**:

Obtain the
data source



Create the
query



Execute the
query

```
// The Three Parts of a LINQ Query:  
// 1. Data source.  
int[] numbers = [ 0, 1, 2, 3, 4, 5, 6 ];  
  
// 2. Query creation.  
// numQuery is an IEnumerable<int>  
var numQuery =  
    from num in numbers  
    where (num % 2) == 0  
    select num;  
  
// 3. Query execution.  
foreach (int num in numQuery)  
{  
    Console.WriteLine("{0,1} ", num);  
}
```

Data Source examples:

- Local list

```
// The Three Parts of a LINQ Query:  
// 1. Data source.  
int[] numbers = [ 0, 1, 2, 3, 4, 5, 6 ];
```

- XML file

```
// Create a data source from an XML document.  
// using System.Xml.Linq;  
XElement contacts = XElement.Load(@"c:\myContactList.xml");
```

- Database

```
Northwnd db = new Northwnd(@"c:\northwnd.mdf");  
  
// Query for customers in London.  
IQueryable<Customer> custQuery =  
    from cust in db.Customers  
    where cust.City == "London"  
    select cust;
```


Query

- The query specifies **what information to retrieve** from the data source or sources.
 - Optionally, a query also specifies how that information should be *sorted*, *grouped*, and *shaped* before being returned.



The query expression contains three clauses:



```
// 2. Query creation.  
// numQuery is an IEnumerable<int>  
var numQuery =  
    from num in numbers  
    where (num % 2) == 0  
    select num;
```

- If you're familiar with SQL, you noticed some similarities.

Query Execution

- **Deferred Execution**

- The query variable itself only stores the query commands. The actual execution of the query is deferred until you iterate over the query variable in a foreach statement.

```
// 3. Query execution.  
foreach (int num in numQuery)  
{  
    Console.WriteLine("{0,1} ", num);  
}
```

- **Forcing Immediate Execution**

- Queries that perform aggregation functions over a range of source elements must first iterate over those elements.
- Examples of such queries are Count, Max, Average, and First.

```
var evenNumQuery =  
    from num in numbers  
    where (num % 2) == 0  
    select num;  
  
int evenNumCount = evenNumQuery.Count();
```

A **query** might do one of **three things**:

- Retrieve a subset of the elements to produce a new sequence **without modifying** the individual elements.

```
IEnumerable<int> highScoresQuery =  
    from score in scores  
    where score > 80  
    orderby score descending  
    select score;
```

- Retrieve a sequence of elements as in the previous example but **transform them** to a new type of object

```
IEnumerable<string> highScoresQuery2 =  
    from score in scores  
    where score > 80  
    orderby score descending  
    select $"The score is {score}";
```

- Retrieve a **singleton value** about the source data

```
var highScoreCount = (  
    from score in scores  
    where score > 80  
    select score  
) .Count();
```

Ending a query expression

- A query expression must end with either a **group** clause or a **select** clause.
- **group** clause
 - Use the group clause to produce a sequence of groups organized by a key that you specify. The key can be any data type.

```
var queryCountryGroups =  
    from country in countries  
    group country by country.Name[0];
```

- **select** clause
 - Use the select clause to produce all other types of sequences. A simple select clause just produces a sequence of the same type of objects as the objects that are contained in the data source.

```
IEnumerable<Country> sortedQuery =  
    from country in countries  
    orderby country.Area  
    select country;
```

- Continuations with **into**

```
// percentileQuery is an IEnumerable<IGrouping<int, Country>>
var percentileQuery =
    from country in countries
    let percentile = (int)country.Population / 10_000_000
    group country by percentile into countryGroup
    where countryGroup.Key >= 20
    orderby countryGroup.Key
    select countryGroup;

// grouping is an IGrouping<int, Country>
foreach (var grouping in percentileQuery)
{
    Console.WriteLine(grouping.Key);
    foreach (var country in grouping)
    {
        Console.WriteLine(country.Name + " : " + country.Population);
    }
}
```

Filtering, ordering, and joining

- Between the starting **from** clause, and the ending **select** or **group** clause, all other clauses (where, join, orderby, from, let) are **optional**.
 - Any of the optional clauses might be used zero times or multiple times in a query body.

- where** clause

```
IEnumerable<City> queryCityPop =  
    from city in cities  
    where city.Population is < 200000 and > 100000  
    select city;
```

- orderby** clause

```
IEnumerable<Country> querySortedCountries =  
    from country in countries  
    orderby country.Area, country.Population descending  
    select country;
```

- join** clause

```
var categoryQuery =  
    from cat in categories  
    join prod in products on cat equals prod.Category  
    select new  
    {  
        Category = cat,  
        Name = prod.Name  
    };
```

- **let** clause

```
string[] names = {"Svetlana Omelchenko", "Claire O'Donnell", "Sven Mortensen", "Cesar Garcia"};
IEnumerable<string> queryFirstNames =
    from name in names
    let firstName = name.Split(' ')[0]
    select firstName;

foreach (var s in queryFirstNames)
{
    Console.Write(s + " ");
}

//Output: Svetlana Claire Sven Cesar
```

- **Subqueries** in a query expression

```
var queryGroupMax =
    from student in students
    group student by student.Year into studentGroup
    select new
    {
        Level = studentGroup.Key,
        HighestScore = (
            from student2 in studentGroup
            select student2.ExamScores.Average()
        ).Max()
    };
};
```

- The following example shows a simple **query expression** and the semantically equivalent query written as a **method-based query**.

```
int[] numbers = [ 5, 10, 8, 3, 6, 12 ];

//Query syntax:
IEnumerable<int> numQuery1 =
    from num in numbers
    where num % 2 == 0
    orderby num
    select num;

//Method syntax:
IEnumerable<int> numQuery2 = numbers.Where(num => num % 2 == 0).OrderBy(n => n);

foreach (int i in numQuery1)
{
    Console.Write(i + " ");
}
Console.WriteLine(System.Environment.NewLine);
foreach (int i in numQuery2)
{
    Console.Write(i + " ");
}
```



```
var cust = new Customer { Name = "Mike", Phone = "555-1212" };
```

```
var newLargeOrderCustomers = from o in IncomingOrders  
    where o.OrderSize > 5  
    select new Customer { Name = o.Name, Phone = o.Phone };
```

```
var newLargeOrderCustomers = IncomingOrders  
    .Where(x => x.OrderSize > 5)  
    .Select(y => new Customer { Name = y.Name, Phone = y.Phone });
```

- **Anonymous** Types

```
select new {name = cust.Name, phone = cust.Phone};
```

Store the results of a query in memory

- A **query** is basically a **set of instructions** for **how** to retrieve and **organize** data.
 - Queries are executed lazily, as each subsequent item in the result is requested.
 - When you use foreach to iterate the results, items are returned as accessed.
- To evaluate a query and store its results **without executing** a foreach loop, just call one of the following methods on the query variable:

```
List<int> numbers = [1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20];

IEnumerable<int> queryFactorsOfFour =
    from num in numbers
    where num % 4 == 0
    select num;

// Store the results in a new variable
// without executing a foreach loop.
var factorsofFourList = queryFactorsOfFour.ToList();
```

- ToList
- ToArray
- ToDictionary
- ToLookup

Delegates

Summary

You can declare delegates using any of the following methods:

- Declare a delegate type and declare a method with a matching signature:

```
// Declare a delegate.
delegate void NotifyCallback(string str);

// Declare a method with the same signature as the delegate.
static void Notify(string name)
{
    Console.WriteLine($"Notification received for: {name}");
}
```

```
// Create an instance of the delegate.
NotifyCallback del1 = new NotifyCallback(Notify);
```

```
// C# 2.0 provides a simpler way to declare an instance of the delegate.
NotifyCallback del2 = Notify;
```

Delegates

Summary

- Declare an anonymous method:

```
// Instantiate NotifyCallback by using an anonymous method.  
NotifyCallback del3 = delegate(string name)  
    { Console.WriteLine($"Notification received for: {name}"); };
```

- Use a lambda expression:

```
// Instantiate NotifyCallback by using a lambda expression.  
NotifyCallback del4 = name => { Console.WriteLine($"Notification receive
```



Lambda expressions

A **lambda expression** can be of any of the following two forms:

- Expression lambda that has an **expression** as its body:

```
(input-parameters) => expression
```

- Statement lambda that has a **statement block** as its body:

```
(input-parameters) => { <sequence-of-statements> }
```

Lambda expressions

Summary

Examples:

```
Func<double, double> cube = x => x * x * x;
```

```
Func<int, string, bool> isTooLong = (int x, string s) => s.Length > x;
```

```
Action<string> greet = name =>
{
    string greeting = $"Hello {name}!";
    Console.WriteLine(greeting);
};
greet("World");
// Output:
// Hello World!
```



All LINQ query operations consist of three distinct **actions**:



The **query expression** contains three clauses:



```
//Query syntax:
IEnumerable<int> numQuery1 =
    from num in numbers
    where num % 2 == 0
    orderby num
    select num;

//Method syntax:
IEnumerable<int> numQuery2 = numbers.Where(num => num % 2 == 0).OrderBy(n => n);
```


Query Execution

- Deferred Execution
- Forcing Immediate Execution

A **query** might do one of **three things**:

- Retrieve a subset of the elements to produce a new sequence **without modifying** the individual elements.
- Retrieve a sequence of elements as in the previous example but **transform them** to a new type of object
- Retrieve a **singleton value** about the source data



- LinqPad - <https://www.linqpad.net/>